M363 Unit 2
UNDERGRADUATE COMPUTING

Software engineering with objects

Requirements concepts

Unit 2

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Edited and designed by The Open University.

Typeset by The Open University.

Printed in the United Kingdom by Thanet Press Ltd, Margate

ISBN 978 0 7492 1608 5

1.1

CONTENTS

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Tara Marshall, Print Procurement

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

In this unit, we introduce the concepts of user and software requirements. We discuss the importance of documenting and managing requirements, and the complexity of the requirements engineering process. Sections 3 and 4 discuss the two main classes of requirement – functional and non-functional – and Section 5 discusses the links between requirements and testing. In Section 6, we introduce a requirements documentation template, which is based on a well known method. *Unit 3* will discuss in detail the techniques used to represent and specify requirements.

In several respects, the approach taken in this unit draws on that taken in Robertson and Robertson (2006).

Figure 1 Dilbert does user requirements gathering

2 Requirements

On completion of this section you should be able to:

- understand the concept and nature of requirements;
- describe the activities that take place in the requirements engineering process.

> The hardest single part of building a software system is deciding what to build. ... No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.
>
> (Brooks, 1987)

The end product of software development is a software system, whose success will be judged by how well it meets its intended purpose. This purpose is defined by the **requirements**. Requirements consist of what needs to be known about a system before you start developing it. This involves descriptions of various aspects including behaviour of the system, constraints on its operation, specifications of properties, etc. The process of reaching an agreed set of requirements is known as **requirements engineering**. This is a complex process that involves diverse interested parties: the people that are going to use the system; those that are paying for it (the **clients**); those that are to benefit from it; and those that are developing it. These are called the **stakeholders** of the system. Discovering who they are, finding out what they want, and documenting it is the purpose of the requirements engineering process.

Requirements engineering is an important activity in software development and the consequences of poor attention to this activity are costly. Many of the commonly cited problems with software products fall into the following categories:

- the system is delivered late and/or is more expensive than initially thought;
- the system does not deliver what the end users want;
- the system is unreliable and has errors.

These problems usually stem from a lack of understanding of the requirements of the system to be developed, and from a lack of clear agreement on what is to be developed. The later problems are encountered, the more difficult they are to solve. It is also a fact that the cost of errors made at the requirements stage is high: any change that needs to be made to requirements will incur much higher costs if detected later in the software development process than if the change were made before design and implementation.

The 1994 CHAOS Report (Standish Group, 1994) identified that, in the United States that year:

- 31% of all software projects were cancelled before they were completed (a waste of $81 billion);
- 53% of projects cost on average 189% of their original estimate;
- in large companies, 9% of projects were on time and within budget;
- in small companies, 16% of projects were on time and within budget.

This survey also asked the participants to identify the causes of these problems, and the top three reasons were given as:

- lack of user input (12.8%);
- incomplete requirements and specifications (12.3%);
- changing requirements and specifications (11.8%).

All these point to the need for a thorough requirements engineering process.

2.1 The nature of requirements

The specification of the requirements for a system is not an easy task. Requirements are not usually stable, and they tend to change as the environment changes; there are usually a wide number of stakeholders, and they do not always have the same view or priorities for the system to be developed; it is not always clear and easy to identify what the requirements for a system are; and there are many factors that influence what the requirements are, and these are not always explicit.

Let us consider the example of a hospital admissions service. The stakeholders are many and with different interests (doctors, nurses, administrative staff and patients). What is intended of the system may not be clear until specific requests have been considered, some of which may not have occurred before. Any change in the organisation of the hospital may affect what is intended of the system; there has to be a consideration of which changes are likely to occur and how they may affect the system. For example, is the division into wards stable, or might wards be split or merged over time? And finally, it is likely that in such a large organisation there are organisational and political factors that will influence the system but may not be clearly stated. For example, there may be conflicts between the interests of doctors and administrators in the way the system is used.

Requirements have to be carefully defined and checked for different properties. Specifically, they need to be:

- necessary and traceable– each requirement should fulfil a purpose, and it should be clear where each requirement comes from;
- non-ambiguous and realistic – there should be no alternative interpretations for a requirement, and it should be feasible to carry each requirement through to development;
- complete – all the intended functionality should be described, although in practice it is often not possible to be absolute about this, and it is important to allow for requirements that emerge later in the system development and during maintenance;
- consistent – requirements should not contradict each other;
- verifiable – it should be possible to check that a requirement has been implemented.

Requirements should also be independent of design, avoiding design decisions that are not relevant at this stage. However avoidance of all design issues is not always practical.

The requirements for a system effectively form a contract between those commissioning the system and the developers of the system. In order that we can later determine whether that contract has been fulfilled, we need to express our requirements clearly and in such a way that we can tell whether the system meets them, i.e. they need to be testable. While we are gathering requirements, we may choose to defer the issue of testability, since it is a concern we can address separately; for this reason, we defer further consideration of this issue until Section 5.

2.2 Requirements dependencies

Requirements have numerous complex and non-trivial dependencies: they often conflict with each other when they make contradictory statements about a system's properties; they may cooperate when they mutually reinforce such properties. For example, consider a requirement which involves a complex request for execution-time resources. This requirement can interact in unexpected ways with other requirements such as performance, security and reliability. In a mobile phone, because of the small screen size, only a few lines of the text can be displayed at one time, requiring the user to scroll up and down to view the text. Thus, the required portability of the mobile phone, and therefore its small size and display area, conflicts with the usability of the phone. So the requirements of usability and portability cannot both be implemented exactly as desired.

Despite well specified requirements, many software projects are vulnerable to failure, or do not meet the stakeholders' requirements. This is because sometimes the dependencies between requirements are not investigated at the requirements engineering stage of the software development life cycle. The importance of this investigation of dependencies can be seen in Example 1 (Boehm and In, 1996).

Example 1

In the New Jersey Department of Motor Vehicles licensing system, engineers were faced with two conflicting goals: low development costs and rapid development versus the need for the system to be able to process licence applications quickly. Unfortunately, the approach they took to ensure low development costs and rapid development produced a system that failed because of performance problems.

A practical approach for investigating the dependencies between requirements is to consider a few representative **scenarios** from a set of all the possible scenarios that illustrate the requirements in practice. By a *scenario*, we mean a specific sequence of activities that might occur when a system is used. For example, in a banking system we might consider the scenario where a user withdraws fifty pounds from their account. Scenarios can be helpful in contextualising the requirements, and for identifying the conflicts and cooperations between them. Unfortunately, there is no formal way to select the 'right' scenarios. The selection is based on an analyst's experience. New requirements for an existing system, and requirements that cut across different parts of a system, are candidates for generating scenarios and investigating the dependencies among them. We will discuss scenarios in detail in *Unit 3*.

2.3 The requirements engineering process

The requirements engineering process, like any other process, takes a set of inputs, consists of a set of activities, and delivers a set of outputs.

The inputs of a requirements engineering process are the information from which the requirements can be derived. As we saw above, the stakeholders' needs are one of the inputs into this process. Other inputs come from existing knowledge of the domain and associated regulations, the organisation's standards, and any systems that are already in place.

The outputs of a requirements engineering process are the requirements document and the models of what the system is intended to do. The requirements document includes the requirements specification containing the precise and accurate descriptions of each requirement; we will look at this document in Section 6.

As for the activities that take place in the requirements engineering process, there are many alternative models to describe them and their ordering. The process may vary depending on the structure and culture of the organisation for which it is developed, and the experience of people using it. There are, however, a set of activities that can be identified as common to many requirements engineering processes, though they may have different names. These are **requirements elicitation**, **requirements analysis** and negotiation, requirements documentation and **requirements validation**.

Requirements elicitation is the activity concerned with identifying the requirements; this is done by consulting the stakeholders, reading existing documents, understanding the domain, defining the boundaries of the system, and understanding the possibilities for change. There are many techniques for elicitation, such as interviewing, brainstorming, running focus groups and holding team meetings. Often several of these techniques are used in conjunction with one another. There are also modelling techniques to help this process, such as the use of **activity diagrams** and **use cases**; we will discuss these in the next unit.

The goal of elicitation is to find out what the system to be developed should do. This involves identifying the system's stakeholders, defining the boundaries of the system, and determining the main objectives the system has to meet. At this stage, we are discovering and recording requirements.

Requirements analysis and negotiation is the activity where requirements are categorised and prioritised, and examined for their properties of consistency, completeness and non-ambiguity. Models of requirements are created to help communication with the customers and developers.

Requirements documentation is usually done using a template document. We use one such template later in this unit.

Requirements validation consists of a careful check of the overall requirements documentation, usually following a checklist of questions. This is a way of ensuring that the requirements that get used by later stages in the software development life cycle are of sufficient quality.

Elicitation and analysis occur primarily, but not exclusively, at the beginning of a project, and their relative importance and effort vary over time. Thus the process of requirements engineering is not generally linear and sequential. At each iteration of the process, activities may be revisited and adjustments made; some of the activities may be carried out in parallel, to some degree.

Figure 2 illustrates the relative effort of requirements elicitation versus requirements analysis in the early phases of a project. Initially, elicitation is dominant. Most of the time, the two activities interact and inform each other: requirements elicitation informs the modelling of functionality and data (which is part of analysis); analysis helps to discover new requirements.

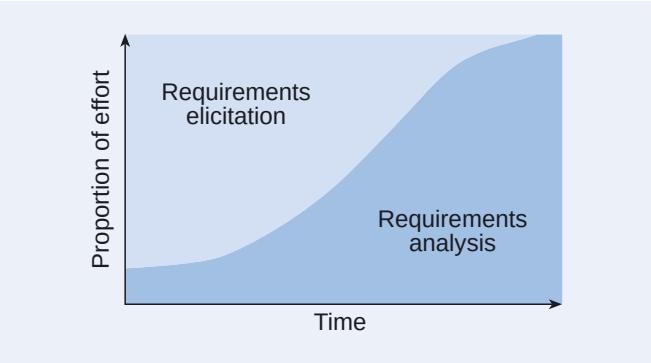

Figure 2 Relative effort of requirements elicitation versus analysis over time

SAQ 1

Where and when does the requirements engineering process take place in an iterative and incremental development process?

ANSWER.....

The main output of a requirements engineering process is the contract between those commissioning the system and the developers of the system. It therefore has to take place early in the software development process. However, an iterative and incremental process recognises that requirements are not stable, and that revisiting, clarifying and specifying requirements occurs in parallel with the other phases of development.

SAQ 2

(a) Identify the stakeholders for a system to book appointments for a hospital.

(b) Suggest ways in which requirements may evolve.

ANSWER.....

(a) Stakeholders include hospital administrators, receptionists, doctors, nurses, patients and the general public.

(b) New requirements may be added. Existing requirements may change because of changes in the environment or in the organisation. Some requirements may become obsolete.

SAQ 3

Consider the following list of poorly expressed requirements. Indicate which of the properties introduced in Subsection 2.1 are not respected, and ask a question to clarify the meaning of the requirement.

(a) The software system should provide acceptable performance under maximum-load conditions.

(b) If the system fails in operation there should be minimal loss of data.

(c) The software should be developed so that it can be used by inexperienced users.

ANSWER.....

(a) The requirement is ambiguous and not verifiable. How can performance be measured? What is maximum load?

(b) The requirement is ambiguous and not verifiable. What is minimal loss of data?

(c) The requirement is ambiguous. What are the usability criteria?

SAQ 4

(a) What are requirements and stakeholders, and how do they relate to each other?

(b) What are the benefits of documenting requirements within a project?

ANSWER.....

(a) Requirements are the functions and qualities we want of a product. Stakeholders are the people and organisations with a vested interest in the product. Requirements arise from stakeholders' needs.

(b) Requirements documentation records decisions; it is the main reference for what should be built, and the basis for validation of the built system.

Exercise 1

(a) Who are the stakeholders in a hotel reservation system?

(b) Consider the example of a hotel reservation system, and invent some examples of the main inputs for a requirements engineering process.

Solution.....

(a) The stakeholders include the hotel owners, receptionists, existing customers and the general public accessing the system to make reservations.

(b) Examples of inputs include:

- stakeholder needs – there should be a help system for first-time users of the system;
- domain information – rooms are identified by a number where the first digit indicates the floor, and the subsequent digits indicate the number of the room on that floor;
- existing documentation – there is a manual system currently used for reservations;
- regulations and organisational standards – an acknowledgement is sent to every customer that makes a reservation.

2.4 Summary of section

In this section, we introduced software requirements, and discussed their relevance to software development, along with the consequences of paying insufficient attention to understanding the requirements for a system. We discussed the importance of involving the various stakeholders in the definition of requirements, and the need to document requirements as a contract for development. We also described the activities that take place in a requirements engineering process.

3 Functional requirements

On completion of this section you should be able to:

- describe functional requirements for a system;
- describe a requirements elicitation process;
- understand the distinction between a technical solution requirement and a business functional requirement.

Functional requirements are those requirements that specify the behaviour of a system. This is in contrast to non-functional requirements, which specify more general properties such as **usability**, **reliability** and **maintainability**. In Robertson and Robertson (2006), the authors define functional requirements as follows:

- specifications of the product's functionality;
- actions that the product must take – check, calculate, record, retrieve etc.;
- derived from the fundamental purpose of the product;
- not a quality – for example, 'fast' is a quality, and is therefore a non-functional requirement.

Another useful description is given in (Alexander and Stevens, 2002), where the authors firmly distinguish between the needs of the user – 'A requirement is a statement of need, something that some class of user or other stakeholder wants' – and what a product (the solution) does: 'A function is something that a system or subsystem does, presumably because a requirement made it necessary. ... A functional requirement ... in practice just means the same as function.'

Here is an example of a functional requirement: 'The system shall accept a credit card number from a client.' Associated with such a requirement are other issues such as a need for the function to be done securely and within a given amount of time. These last two requirements are traditionally considered to be non-functional requirements. They constrain a functional requirement to being done in certain ways. However, the distinction between functional and non-functional requirements is to some extent pragmatic rather than fundamental, since, for example, security is very much associated with functionality. Whether or not you are convinced that there is a real difference between functional and non-functional requirements, we will continue with the distinction, as it helps in drawing up check lists of places to look for requirements.

3.1 Elicitation and analysis and negotiation

When eliciting functional requirements, you should pay attention to their relative importance. It is important to identify those requirements that are essential (often said to be 'core requirements') and those that are inessential but would be nice to have. The reason is simple: more requirements tend to increase both the cost and the time for development. So by identifying the core requirements, you should be able to identify the minimum cost and time for development. More generally, it is felt that functional requirements should be prioritised, so that the most important requirements can be identified. This enables a judgement to be reached on what can be delivered for a given price, or what the cost will be for a given set of requirements.

In practice, when eliciting requirements, you would probably use a way that best suits your experience and applications, and that results from a combination of different

approaches. The specific approach that we use in this course for eliciting requirements is described in detail in *Unit 3*, but in outline is as follows.

We start by looking at the domain – the current business situation – that is, how the work is carried out before any development takes place. The domain modelling activity will identify a set of **business processes** triggered by **business events**. Once a decision is taken to develop a system, these business processes and events become the context for the system being developed. Some of the business events will be directly relevant to the new system; each of these events will then be associated with a use case – a chunk of functionality – that is, a description of how the new system is to be used for a specific goal.

Use cases are derived from business events, and each use case is described by a set of scenarios. A scenario is a series of steps that complete the functional tasks of a use case, described from the point of view of a user. A task is something that the actor identifies as being part of the work of the use case. Thus the steps in a scenario provide a mechanism (thinking technique) for determining all the functional requirements needed by each step.

In *Unit 3*, we discuss use cases in great detail, and use a template to represent them and their associated scenarios.

One way to discover the requirements of a system is to see what the system needs to do to support each step in a use case. For example, in a scenario for checking a guest into a hotel, there is a step 'allocate room'. This step maps directly to the requirement 'The system shall allocate a room'. On the other hand the step 'issue key to guest' might be a completely manual operation, and is unlikely to involve any software requirements. For complex tasks or steps, it may be that several requirements are involved. One advantage of deriving requirements from use case descriptions is that we can then associate the requirements with the specific goal of the use case to which they relate; this is a way of structuring requirements. As we work through the use cases of a system, we are likely to find requirements that are repeated, that is, a requirement may belong to several use cases. Identifying these common requirements will allow us to implement them just once.

In reality, of course, the elicitation of requirements and the scoping of the system starts on the basis of an initial set of quite general, high-level requirements, for example, 'accept a reservation' or 'check a guest in'. So the process described in the last paragraph can be seen as a way of obtaining the detailed requirements of the system.

We have finished collecting requirements once they constitute a description of everything the system must do, that is, once the system can be built without further reference to the client or users of the system. This does not mean of course that the system will not be constructed until all the requirements are gathered: in an iterative approach quite the opposite will be the case. Furthermore, it is likely that requirements will continue to emerge long after the system has to be installed. It is therefore crucial that requirements are carefully managed, and failure to do so is one of the most important causes of project failure. One approach to prioritisation, used in the DSDM (Dynamic Systems Development Method) framework, is the MoSCoW scheme, where requirements are classified as 'Must haves', 'Should haves', 'Could haves', and 'Won't haves'. This classification scheme can be used at the level of each iteration, allowing an iteration to be timeboxed. That is, when the time allocated runs out, 'should have' and 'could have' requirements are delayed to a later iteration, rather than adding time.

For more details, see http://na.dsdm.org/en/about/moscow.asp

3.2 Describing requirements

In writing requirements, we are looking for clear statements of the form 'The system shall ...'. In doing this we need to be careful to avoid **ambiguity**, which is not easy, because natural language is notoriously ambiguous. For example, consider the

requirement 'The system shall allocate a room, and it should be cheap'. By using the word 'it', we have introduced ambiguity: is it the room or the system that should be cheap? (Furthermore, what does cheap mean?) Pronouns ('it', 'their' etc.) should be avoided. Another source of ambiguity is the way in which a word may have multiple meanings, or, even where it has essentially one meaning, can be interpreted in different ways by different people. For example, are the users of a library system the library's members, its staff or both? For this reason, any words that have technical meanings or meanings specific to the business should be given definitions that get recorded in the requirements specification. For similar reasons, abbreviations (including acronyms) should be defined there.

Statements may also need to be constrained in other ways. For example, if we have to deal with VIP guests, we may need a requirement such as 'The system shall allocate a deluxe room'; the simpler statement 'The system shall allocate a room' is too loose.

Requirements should be described at a level of detail that the people commissioning the system can understand. Furthermore, it is useful to write descriptions of requirements in such a way that 'the user should be able to tell you whether there is enough functionality for the product to be useful, and whether the functionality is correct given the intended outcome of the work' (Robertson and Robertson, 2006). However, such descriptions may not be sufficiently detailed for developers. The question arises, therefore, as to what is the appropriate level of detail to be provided by a requirements analyst.

Ian Somerville distinguishes two types of requirement, as follows.

> Requirements specification [is] the activity of translating the information gathered during the analysis activity into a document that defines a set of requirements. Two types of requirements may be included in this document. **User requirements** are abstract statements of the software requirements for the customer and end-user of the system; **software requirements** are a more detailed description of the functionality to be provided.
>
> (Somerville, 2004)

We need to work with both types of requirement. We need user requirements when interacting with stakeholders external to the project, and software requirements to inform system design. The two forms of requirement are related, as the software must be designed to meet the user requirements. The first type of requirement is usually written in natural language, for obvious reasons. Either natural language or a more formal approach (for example mathematical specifications) can be used for software requirements. In both cases, our requirements should be recorded in a structured document (you will see more about this later in the unit).

It is important to distinguish between the requirements and the systems that we introduce in order to satisfy them, that is, produce the required behaviour: there is a major distinction to be made between a requirement and its solution. For example, we might have a requirement 'The system shall produce a list of names in alphabetical order'. It would not help us to specify this as 'The system will sort a list of names using the bubble-sort algorithm'. You must ensure that you do not write solutions instead of requirements. You need to understand the essence of the business, not its implementation. This is important because it is too easy to hide important functionality by describing an implementation, and too easy to select one implementation when better ones may exist.

Another issue that arises because of solution considerations is that of **technical solution requirements**. These relate to functionality that is needed purely because of the chosen technology. Technical solution requirements are not there for business reasons, but to make the chosen implementation work correctly. They are constraints

on the behaviour of the product. It is essential to ensure that the technical requirements are not introduced before the business requirements have been fully understood. For example there may be a need to authenticate the user of a system. This is a business requirement. However, if we state that this should be done in a particular way, for example, using a password (as opposed to, say, an iris scan), this is a technical solution requirement.

SAQ 5

(a) What are the purposes of a requirements document?

(b) What is a functional requirement?

(c) What is one property that a functional requirement should not possess?

(d) What is a non-functional requirement?

(e) What is a technical solution requirement and what is a business functional requirement? Why is it useful to distinguish them?

(f) What overarching property should the set of functional requirements that result from a requirements gathering process possess?

(g) How do business events and use cases help in determining functional requirements?

ANSWER.....

(a) A requirements document is for communication – from the requirements engineer to the designer. It serves as a contract with the client (and other stakeholders) for what the system must do.

(b) A functional requirement describes an action that the product must take if it is to carry out the work it is intended to do.

(c) A functional requirement should not be a statement about a quality.

(d) A non-functional requirement is a requirement about a quality that the product must have.

(e) A technical solution requirement is a constraint on the product due to the technology of the solution that must be adopted.

A business functional requirement is a specification of the work, or business, independent of the way that work will be carried out.

Thus the two types of requirement arise from different domains: the business domain and the solution domain. We want to keep issues related to the business separate from those of the solution.

(f) The set of functional requirements must fully describe the actions that the intended product can perform. That is, the product's builder must be able to construct the product desired by the client from the descriptions contained in the functional requirements.

(g) Use cases are derived from business events, and each use case is described by a set of scenarios. A scenario is a series of steps that complete the functional tasks of a use case. A task is something that the actor identifies as being part of the work of the use case. Thus the steps in a scenario provide a mechanism (thinking technique) for determining all the functional requirements needed by each step.

SAQ 6

(a) How do you discover whether a set of functional requirements are sufficient for the product to be useful, and whether the functionality is correct?

(b) Why must functional requirements be testable?

(c) Can you think of some generic questions to ask that can help in making requirements precise and complete?

(d) What is the major problem with a requirement that is written in a natural language such as English?

(e) Is it possible or desirable to avoid ambiguity entirely in a requirements specification? What steps can you take to reduce ambiguity in a requirements specification?

(f) Why should you record the meanings of business and technical words in a requirements specification?

ANSWER.....

(a) Ask the user!

(b) It can then be determined whether the delivered product meets the intention of the user.

(c) Questions of the form 'When should something happen?' and 'To whom should something be sent?' are useful. You may have thought of others.

(d) Natural language is often ambiguous.

(e) It may be possible to avoid ambiguity entirely, but the cost of being so precise can be enormous and the result unreadable. Therefore, we accept having to live with some ambiguity, but try to ensure that the risk of doing so is acceptably small. One way in which you can try to reduce ambiguity is by providing clear definitions of any technical terms you use. A second way is by ensuring that requirements are not stated in too general terms.

(f) They should be recorded to avoid ambiguity and aid clarity in the usage of terms.

SAQ 7

Summarise the overall process described above for determining a set of functional requirements.

ANSWER.....

1 Understand the domain, determining the business processes and business events.

2 Determine the scope of the new system, and which business events are relevant.

3 Draw up a set of use cases for the product associated with those events.

4 Describe each use case by one or more scenarios – sets of steps.

5 Work through each step of each scenario to determine a set of system requirements.

6 Check for similar requirements from different use cases.

7 Search out and remove ambiguity.

Exercise 2

Here is a list of requirements that apply to the product *X*. For each requirement, say whether it is a functional or non-functional requirement, and if it is a functional requirement, say whether it is a business requirement or a technical solution requirement.

(a) *X* must check a user's identity.

(b) *X* must check a user's password.

(c) *X* must produce a statement of the user's account.

(d) *X* must validate the user's identity and password within 3 seconds.

(e) *X* must be usable by users with limited dexterity.

(f) *X* must employ the company's proprietary password-maintenance system.

(g) *X* must operate in arctic climates.

Solution .....

(a) Functional, business requirement (the requirement states what needs to be done, not how to do it).

(b) Functional, technical solution requirement (access to the system can be achieved in many ways; passwords are just one mechanism, but biometrics may be more appropriate to this product).

(c) Functional, business requirement (the requirements states what needs to be done, not how to do it).

(d) Non-functional.

(e) Non-functional.

(f) Functional, technical solution requirement (a specific technology is mandated).

(g) Non-functional.

Exercise 3

Derive some functional requirements for the following step of a scenario in a hotel management system: 'The system shall allocate an available cleaner to each occupied room.' Confine yourself to what must be done, not how it is to be done.

Solution .....

Here are some suggestions.

- The system shall identify occupied rooms.
- The system shall identify available cleaners.
- The system shall allocate occupied rooms among the available cleaners while balancing their workloads.

Exercise 4

Imagine that you have been commissioned to produce a Requirements Recording Tool. The purpose of the tool is to enable a requirements analyst to record information about each requirement for a product. An important requirement for the tool is that it should give its users the impression that they are dealing with a set of cards, each recording a requirement.

Without considering the detail of the information to be kept about each requirement, identify four functional requirements for managing a collection of such records. That is, what could the tool to do to help you record the requirements you have elicited so far, and maintain those requirements as you learn more about the required product? If the tool is to be used on different projects, what additional functionality should the tool possess?

Solution .....

Here are some of the functional requirements that relate to the management of a set of requirements. Yours may differ.

- The product shall enable a new requirement to be entered.
- The user shall be able to view each requirement in the set.

- The product shall enable a user to amend (edit) an individual requirement.
- The product shall enable the user to delete a selected requirement from the set of requirements.
- The product shall enable the user to view a summary of all requirements.
- The product shall show all requirements that conflict with a given requirement.
- The product shall perform a range of checks on the set of requirements.
- The user shall be able to view all requirements satisfying a given criterion, for example, those that mention particular terms.
- The product shall show a list of all requirements associated with a selected business event.
- The product shall enable the user to create and amend a set of requirements for a new product.

3.3 Summary of section

In this section, we defined functional requirements, and illustrated a process for eliciting functional requirements. We discussed the issues surrounding the documentation of requirements, and distinguished between technical solution requirements and business functional requirements.

4 Non-functional requirements

On completion of this section you should be able to:

- describe non-functional requirements;
- illustrate and characterise the different types of non-functional requirement;
- discuss the main characteristics of security requirements;
- identify the main issues raised by security.

Functional requirements state what a product must do. In contrast, non-functional requirements are qualities that the system should have, such as being secure, fast, usable, maintainable etc. So, for example, while we may have a requirement that a banking system should compute and display a client's balance, we may also have non-functional requirements attached to this, such as that it should be done securely, and within, say, five seconds. Non-functional requirements might be associated with a particular piece of functionality (a use case or group of use cases) or with the overall system. In the latter case, we would probably need to break down what is meant in some instances; for example, given a non-functional requirement that 'the system shall be secure', we probably need to know what this means in terms of individual use cases, as well as for the overall system. On the other hand, a non-functional requirement like 'all responses from the machine shall complete in one second' can be applied globally with little further clarification.

One useful way to think of non-functional requirements is as constraints on the functional requirements. From this perspective, functional requirements tell us what the system should do, and non-functional requirements constrain the ways in which those things are done: securely, quickly etc. This means that, in effect, non-functional requirements only make sense in relation to what they say about functional requirements. Functional requirements make more sense in isolation, but even so, they do not give us the full picture of what we need to develop.

In recent times, requirements engineers have begun to change their terminology from 'non-functional requirements' (or 'NFRs' for short) to 'quality requirements', simply because the new term is more suggestive and less confusing than the original. However, as this course was being written, both terms as well as some others were in common usage.

Non-functional requirements are elicited by working from a list of categories, and deciding whether they apply to each requirement and use case, and if so, in what way. We will explore such categories in this section, and a list of them is used in the requirements template introduced in Section 6.

Perhaps the most surprising aspect of non-functional requirements is that they not only enable the functionality of the product, but they can also add up to a significant part of the specification. The significance can be in terms of size (the proportion of the specification dealing with non-functional requirements) and perception (how the users perceive the product – its usability and appearance, for example). No matter how well a product does its work, whether or not people will use it can depend crucially on its quality aspects.

Previously, it was generally felt that non-functional requirements were difficult to deal with because they were too 'nebulous'. Today, we view non-functional requirements in the same light as functional requirements: to be useful, requirements must be testable,

We use the terminology of *fit criteria* from Robertson and Robertson (2006).

which means that they must be measurable. Precisely how we make non-functional requirements measurable is dealt with in Section 5, which introduces the idea of **fit criteria**. You can regard a fit criterion as a quantification or measurement of a requirement that will ultimately enable a tester to determine whether or not the delivered product satisfies, or fits, the requirement.

In *Unit 1*, we mentioned the role of architecture. Non-functional requirements constrain us in our choice of architecture, in that the chosen architecture must allow us to achieve the qualities, such as speed and maintainability, that we require.

In Robertson and Robertson (2006), the authors identify the following eight classes of non-functional requirement:

- look-and-feel requirements – the spirit of the product's appearance;
- usability requirements – the product's ease of use, and any special usability considerations;
- performance requirements – how fast, how safe and how accurate the functionality must be;
- operational requirements – the operating environment of the product, and what considerations must be made for this environment;
- maintainability and portability requirements – expected changes, and the time allowed to make them;
- security requirements – the security and confidentiality of the product;
- cultural and political requirements – special requirements that come about because of the people involved in the product's development and operation;
- legal requirements – the laws and standards that apply to the product.

While the above list represents a useful way of categorising non-functional requirements, it should not be thought of as exhaustive, and depending on context, other ways of categorising requirements may be more appropriate.

We will now look at each of the above categories, in varying amounts of detail, considering how each can be recorded. We will give particular attention to security requirements, as at the time of writing, this area has attracted a lot of attention, and they constitute a particularly interesting example of how to deal with non-functional requirements.

4.1 Look-and-feel requirements

Look-and-feel requirements are about the overall impression a product will make on a user, that is, the appearance and behaviour of its interface. For example, in a banking environment, we might want the product to have a conservative feel, with limited use of colours and animation. If the product is a game, then the opposite might apply. These requirements are not about the specifics of the user interface, so they are often described in quite general terms. An exception to this is when the product will run on an operating system, and we decide to conform to the detailed user-interface guidelines provided by the manufacturers of the operating system.

The following are examples of look-and-feel requirements.

- The product shall comply with the Motif user-interface guidelines.
- The product shall use only two colours.
- The product shall use lots of animation.
- The product shall use a large range of exciting sounds.

SAQ 8

(a) What does the phrase 'look and feel' refer to?

(b) When identifying non-functional requirements for the look and feel of a product, why should you avoid the temptation to provide a design for the user interface?

(c) The description of a look-and-feel requirement is often loosely worded and therefore difficult to turn into a good design. What should be done to rectify this situation?

(d) What general characteristics should the look and feel of a consumer product have?

ANSWER.....

(a) Look-and-feel requirements describe the overall appearance and behaviour of the product to its users.

(b) The production of a design is the task of the product's designers, once they know the requirements. The look and feel are not about the specifics of the user interface.

(c) Fit criteria (dealt with in detail later in the unit) should be added to the requirements, to make them measurable.

(d) The look and feel is concerned with the *impression* we wish to make; we want it to reflect the distinctive values, ethos and style of our organisation.

4.2 Usability requirements

One of the key criteria for measuring the success of a product is whether users are able to use it to fulfil their tasks. A product with poor usability may result in poor productivity, high error rates due to 'misuse', and high stress levels for users. In some cases users will refuse to use the system. To specify usability requirements, we need to think about the types of people who will use the system. For example, do they have lots of experience with using the type of system we intend to produce? Usability can be split into two issues: how 'easy to learn' is the software and how 'easy to use' is it.

The requirement that a product be 'easy to learn' is contextual. An easy-to-learn website for buying books might require zero training; an easy-to-learn accounts package might require a two-hour tutorial rather than a week of on-site training; an easy-to-learn nuclear-reactor control system might require several months of training but not several years of experience.

An easy-to-use product is designed with a view to long-term efficiency, and may require users to be trained before it can be used effectively. Such a product might be complex and take some time to learn, but the cost of training is outweighed by the resulting efficiency gain.

Easy-to-learn products, on the other hand, are often aimed at those tasks that are performed infrequently and so have to be relearnt from time to time; these products are often targeted at the public market. When training is infeasible, products have to be easy to learn and are not normally complex.

The following are examples of usability requirements.

- A university graduate should be able to learn to use 50% of the functionality of the product in 2 hours.
- 90% of the population should be able to place an order from a web interface within 5 minutes.

SAQ 9

(a) What do usability requirements describe?

(b) What are the effects of usability on a product?

(c) How might you express a usability requirement more precisely than simply 'easy to learn'?

ANSWER.....

(a) Usability requirements specify how easy to use a product should be for its intended users under specified circumstances. They include requirements for how easy it should be to learn to use the product.

(b) Usability impacts on productivity, error rates, stress levels and acceptance. It determines how well the human part of the system can perform.

(c) A usability requirement can be expressed more precisely by describing the level of achievement required after the required training or learning period.

4.3 Performance requirements

Performance requirements relate to the capacity of a system to carry out its tasks. This includes the speed with which the system should operate, how much data or information it can handle, how accurately in can produce its results, and how reliable it is.

Note that the apparent speed of a system can be influenced by the number of transactions or amount of data it is able to process in a given time (known as throughput and volume respectively). A system may be able to deal with a single transaction with the requisite speed, but if the volume of transactions increases, the system may not be able to deliver the same performance.

Reliability is the ability of the system to behave consistently in a user-acceptable manner when operating within the environment for which it was intended. That is, the system should operate correctly and do the same thing each time under the same conditions.

The following are examples of performance requirements.

- The product shall calculate a guest's bill in 2 seconds.
- The product shall handle up to 10 users simultaneously.
- The product shall report wind speeds within 5 miles per hour of the actual speed.
- The product shall, on average, operate without failure for 20 days.

SAQ 10

(a) What are the main kinds of performance requirement?

(b) Rather than accept requirements which state that some thing should be done speedily and/or efficiently, what should you aim for?

ANSWER.....

(a) Normally, the main performance requirements involve speed (the time to do something), capacity, reliability and accuracy.

(b) Look for requirements which specify the speed and efficiency in ways that can be measured objectively.

4.4 Operational requirements

Operational requirements concern the environment in which a product is to be used. For example, imagine a system designed to coordinate a mountain rescue team. While the main system might be located in an office, it might be that the rescue team will interact via portable devices, and these will therefore need to deal with extremes of temperature, humidity and available light. Furthermore, the ability to use the devices will also be affected by these conditions. Another area that can easily be overlooked concerns those requirements that relate to the installation of the product and the needs of the installers.

One approach to finding the operational requirements is to investigate the product boundary (the scope as defined by the use cases), and consider the needs of each adjacent system.

The following are examples of operational requirements.

- The product shall be usable at altitudes up to 1500 m, in icy and wet conditions, and in both darkness and extremely bright sunshine.
- The product will be used in a standard office environment, except that high levels of background noise may occur.
- The product will need to be installed at 58 locations around the proposed route of the race in 2 days by 3 semi-skilled workers.

SAQ 11

How do operational requirements differ from performance requirements?

ANSWER.....

Operational requirements describe the operational environment (factors external to the product) in which the product must function correctly, whereas performance requirements deal with issues such as speed and size (factors internal to the product).

4.5 Maintainability and portability requirements

There are two quite different notions of maintaining a product. The first is to do with keeping the product updated in the light of expected changes. For example, we may know that new requirements are likely to occur at some point in the future. This may be because the way a business operates is likely to change, or because the laws which apply to the business area are to be updated. The second notion of maintainability is to do with mending the product when it fails. Portability requirements concern the need to port the software to new hardware/software environments at some later stage.

The following are examples of maintainability and portability requirements.

- The product shall be able to be modified to cope with a new class of user.
- The product shall be able to be modified to cope with minor changes to European law that occur every six months on average.
- The product shall be portable to all of the operating systems currently used in our Slough office.

4.6 Cultural requirements

It may be quite obvious to you that if a product is to be sold in a different country to your own, there will be potential cultural requirements to consider. But what might they be? Less dramatic, but still important, are the cultural differences between organisations. If you are eliciting requirements for an organisation that is not your own, be wary of cultural differences; do not assume that you will observe the same reaction to your investigations that you might in your own organisation. A way to overcome these problems is to seek the help of someone who is knowledgeable about the culture.

The following is an example of a cultural requirement.

> The language used in the interface should be formal and polite.

4.7 Political requirements

In Robertson and Robertson (2006), the authors classify a political requirement as 'a requirement with no justification other than "I want it"'. This is a somewhat simplistic view. Pugh (1993) points out that all organisations are at the same time both rational and political. A political requirement may be as well justified as, and even more important to the project than, any other requirement – for example, 'the product shall have the full support of the unions.' People may say 'because I say so', but they usually mean 'because of various factors that I am unwilling or unable to tell you about'.

As a requirements analyst, you are more likely than other members of a software project team to encounter the political side of organisational life. For example, when resources are allocated through complex and ambiguous decisions, you may find it difficult to gain access to certain stakeholders when eliciting requirements. In such cases, the project manager may be able to help. While some stakeholders, such as the client, will have the power to get certain requirements included because of their position, others will resort to politics to look after their interests.

SAQ 12

(a) When do cultural requirements usually arise?

(b) What is the best approach to dealing with cultural issues?

(c) What characterises a political requirement?

(d) Why are cultural and political requirements often difficult to deal with?

ANSWER.....

(a) Cultural requirements usually arise:

when a company attempts to sell a product in a different country, particularly a county with a very different culture and/or language from the one that the product was initially designed for;

when eliciting requirements in an organisation different from one's own.

(b) Obtain the help of stakeholders from that culture.

(c) A political requirement is a requirement for which someone is unwilling or unable to provide a coherent rationale. A political requirement is often stated in terms of 'I want it'.

(d) Cultural and political requirements often involve having to ask personal questions, and can be difficult to quantify. Such questioning is likely to be very sensitive.

4.8 Legal requirements

Two important areas from which requirements must be elicited are the law and the standards that apply to the product. Two important pieces of UK law for which there are likely to be equivalents in other countries are the **Data Protection Act** and Health and Safety regulations. When developing computer systems, you must satisfy yourself that the relevant laws will be complied with, either by examining the legislation or, preferably, by asking a lawyer. Failure to comply with the law might involve a range of penalties. In doing this, it is important to look at the environment within which the system will operate, and what other systems and people it will interact with.

A third area related to law is the regulatory structure that controls some industries. For example, companies that make computer-based medical devices must have their equipment approved by the relevant regulatory authority (this is often described as 'regulatory compliance') before it can be used in a medical setting. There may be sets of criteria that the equipment has to satisfy. Indeed, the companies may have to show how the software was developed, and hence meet a number of traceability requirements. Thus the regulatory framework can add a large number of requirements to every such product. For a company working in an international market, the consequences can be even more severe, because each country is likely to have different and possibly conflicting regulations! At the time of writing (2006), the EC is considering introducing mandatory accessibility compliance standards.

As well as ensuring that systems under development comply with all relevant laws, there is also a need to ensure that any contracts entered into with clients are honoured. The cost of litigation is one of the major risks for commercial software, and can be expensive for other kinds of software. Furthermore, the way in which a contract is drawn up plays a strong role in determining whether the software development subsequently undertaken is profitable and reflects well on the professional standing of those involved.

SAQ 13

(a) What is the most pressing reason for considering legal requirements?

(b) How should you determine the appropriate law that affects the product?

ANSWER.....

(a) The cost of litigation is one of the major risks for commercial software, and can be expensive for other kinds of software. There are penalties for non-conformance with the law: fines, imprisonment and loss of reputation.

(b) Obtain help from the company's lawyers.

4.9 Security requirements

This section provides a general introduction to issues of security and protection. It introduces some of the threats that face computer systems. Here we are especially concerned with the issues surrounding computer security and some of the solutions that have been adopted to solve the problems that arise. Security is also a good illustration of conflicting requirements. You can make a system very secure to deter unauthorised access, but it may be at the cost of making access by legitimate users extremely difficult.

Security is an area in which a great deal of research is currently being pursued, and we will deal with it in more depth than other non-functional requirements. However, you can treat the material in this section as an example of how practitioners are approaching the generally difficult subject of non-functional requirements.

Security covers confidentiality, integrity and availability (often abbreviated to CIA), and is concerned with threats against assets. *Confidentiality* refers to the protection of data from unauthorised access and disclosure; *integrity* refers to consistency of data; and *availability* refers to access by authorised users.

As well as the costs incurred if data is lost as a result of its integrity being compromised, there may be costs as a result of data being unavailable, as work cannot then be completed, and contractual arrangements may not be met. Substantial costs are also associated with loss of confidentiality, as commercial secrets may be exposed, individuals may sue, and an organisation's reputation may be harmed in a way that loses it future business. For example, a bank's reputation for confidentiality is critical to its ability to attract and retain customers.

There is a need to protect a computing system and its resources from unauthorised access by those who seek to gain some advantage or act maliciously. There are intruders who try to read, change or delete the data that is stored in, processed by or passed around a computing system. They may attempt to carry out an unauthorised transaction, passing themselves off as another person, and may extract financial gain or advantages through such use. At the extreme, there is a chance of physical damage or destruction.

Some examples of intruders are:

- 'hackers' who test their skills against the security measures of a system for their personal pleasure;
- competitors who may try to gain access to commercially secret information;
- fraudsters who try to obtain financial gain from the owner of the system or some third party.

There are many definitions of security, but we will define computer security as being concerned with the detection and prevention of unauthorised actions by users of a computer system. For example, we might allow (authorise) everybody in the world to read pages on our website. But only a limited number of users would be allowed (authorised) to change or delete those pages.

With a standalone computer, you could achieve security by physical means. For example, you could place the computer in a room to which only certain people have (physical) access. In a simple version of this scheme, those who have access to the machine would have the power to perform any action on the machine. Alternatively, you may wish to restrict the actions of individual users. You might do this by having a mechanism whereby the user has to provide a name and a password for identification, and rely on the operating system to restrict individuals to certain actions (accessing data, running particular applications, and so on). These approaches are, of course, solutions to the problem of security, not requirements.

With a distributed computing system, which involves a number of interconnected computers, there is the possibility of someone being able to intercept users' communications. This interception may be passive (just listening to the communications) or active (listening and retransmitting the messages with or without changes). On an external network, such as the internet, communications pass via a number of computer systems over which the user has no control. In particular, the user does not know what security mechanisms they have in place. Thus security takes us beyond the basic protection of resources to consider the computing system in its surroundings (or environment).

In a distributed computing system, security becomes a major issue. Although communication and sharing are the motivations for forming a distributed computer system, you might not want to share your resources with everyone! The same is true for communication.

Ecommerce is a field in which security is a major issue. We define ecommerce to be a distributed computing system (often using the internet) where commercial transactions take place. Such transactions might be the sale of goods or services or banking services.

You can devise mechanisms to stop casual attacks from pranksters or disaffected employees, but you will not deter everyone. Ultimately, no computer system can be completely secure from attacks, but we can make it very difficult for would-be attackers to succeed, by making the cost of mounting a successful attack very high in terms of time, money, equipment and people.

In the next section, you will look at the different kinds of security threat to a distributed computing system, and then examine what can be done to alleviate them. A single computing system faces the same threats, but at a lower complexity than a distributed computer system.

Threats and attacks against assets

For any kind of computing system, you want to ensure that all resources (also known as assets) are accessed and used as intended under all known circumstances. If you can prevent misuse, the computing system is secure. However, the cost of making a system secure must be balanced against the difficulties the security measures impose on legitimate users.

The forms an attack might take are as follows.

- *Disclosure* (of confidential information) or the unauthorised release of information. For example, an intruder might intercept the network and read your messages en route. From an ecommerce transaction, such as an electronic order you have placed, an intruder might be able to access your credit-card details and illegally use them.
- *Modification* (loss of integrity) or the unauthorised alteration of data (information). Intruders may change messages or stored data. They might increase their bank balance (either by directly modifying the data or by altering a message so they become the payee of an online fund transfer), withdraw all the funds, and then disappear.
- *Denial of use or service*, where there is some denial of network service to its authorised (legitimate) users. The intruder redirects your messages or creates congestion by recycling messages. They may also covertly gain control of the computers owned by innocent third parties and use these computers to bombard your site with messages. This might be a competitor who is trying to put you out of business by preventing access to your system by your customers.
- *Repudiation*, where a legitimate user claims that they did not send or receive a particular message that was sent or received. For example, suppose somebody ordered holiday insurance online, and after the holiday claimed that they did not order the insurance. The insurer could do a similar thing in the event of an insurance claim, repudiating the customer's assertion that they ordered insurance.

Example 2

Suppose that a university used a distributed computing system for all its students, to support distance learning. If all course and assessment material were held within such a system, you would envisage the following security problems:

- disclosure of personal information;
- disclosure of educational materials to people who have not paid for them;
- disclosure of students' assignment solutions to other students;
- unauthorised modification or alteration of data;
- unauthorised changes made to educational materials;
- unauthorised alteration to students' marks;
- denial of use or service;
- inability to access material by students;
- very slow replies to service requests;
- repudiation;
- inability to verify that a student has submitted a particular assignment.

There is no single solution to these and other potential problems. A variety of mechanisms need to be applied, which may affect the operational efficiency of the university in Example 2. Within one organisation, a security policy, or set of rules, would govern the choice of mechanisms for use in response to particular threats. In a well defined network that contains a known set of access points, a security policy may be straightforward. However, a single policy may not be possible in an interconnected network where there can be multiple ownerships of resources and the number of potential access points increases the security threat.

In general terms, a distributed computing system may consist of a number of domains, each one separately managed, operating different security policies and diverse security mechanisms.

SAQ 14

(a) In the context of a computer system, what is meant by security?

(b) There are two problem areas for a distributed computing system that go beyond the normal security requirements. What are they?

(c) From the point of view of a security administrator, suggest a useful starting point to monitor potential threats.

ANSWER.....

(a) Security is about the prevention of unauthorised access to the system.

(b) The additional security problems that arise with a distributed system are that:

- the communication medium is insecure: users' communications may be intercepted en route and read or altered;
- on an external network, communications will pass through many third-party systems with unknown security measures, which cannot be controlled.

(c) One useful focal point is at the boundary of the security domain for which you are responsible. In practice, this is likely to be a firewall for a protected network.

SAQ 15

(a) What are the three aspects of security from a requirements perspective?

(b) Distinguish between the three aspects of security.

(c) Why is it a good idea to consider bringing in a security expert to help identify security requirements?

ANSWER

(a) The three aspects are confidentiality, integrity and availability (CIA).

(b) Confidentiality – data must not be made available to anyone except authorised users. This implies identification of those who are authorised to access specific items of data.

Integrity – the data held by the system corresponds to the data supplied to the system. Integrity implies that data does not become corrupted.

Availability – authorised users of data should not be prevented from or unnecessarily delayed in accessing that data. This implies that steps should be taken to prevent loss of data and to prevent denial of service attacks.

(c) Security is an extremely important issue, and getting it wrong may be disastrous. Where developers have not been trained in security, it would be prudent to have a security expert on the team to advise them.

Exercise 5

Consider a patient-monitoring system in a hospital. What do you think would be a suitable user requirement for the availability of such a system? Does this requirement imply any others? How might you measure its availability?

Solution

The first thing to ask about a patient-monitoring system is: who are the users? Inside a hospital, the doctors and nurses will be responsible for monitoring the patients' health. So they, not the patients, will be the users (although patients certainly have an interest in the reliability of such systems). A suitable requirement might be: 'The system should be unavailable for no more than 6 hours per year.'

Perhaps this also indicates another requirement: 'It must be obvious when there is any fault.' Notice that we did not write something like 'When the system fails it should sound an audible warning', as this would be an unnecessary constraint which is not actually part of the requirements, even though it might well form part of the eventual solution.

You might measure reliability by one or both of the following:

- the proportion of time that the system is operating satisfactorily;
- the number of false alarms.

Exercise 6

A company sells computer accessories (paper, printer ink, modems etc.). It wishes to introduce an ecommerce system to enable customers who have credit accounts to use the internet for their purchases. Some customers pay cash with their order, and some order on credit. Goods that are ordered on credit are sent with an invoice to the customers, and payment is required later. Not all of the customers have credit accounts, and for those that do, there is a limit to the amount of credit they can have. Customers have a customer account number to identify themselves. What security issues are involved in the proposed ecommerce system?

Solution

The main security issues are as follows.

- Authentication is needed to establish the identity of the principal user, that is, the customer placing the order.
- Authorisation is needed to check that the customer making the credit order has a credit account and is authorised to order on credit, and that the amount of the order does not exceed the credit limit.
- Repudiation by a customer who claims not to have placed the order may occur. The company might similarly try repudiating an order it received.
- Disclosure of confidential information used in the transaction for authentication, customer details and the contents of the order itself may occur.
- Integrity of the order must be assured. For example, the quantities of items must not change.

Not all of the issues would necessarily be addressed by having a component in the network or computer systems to manage them. For example, repudiation of the order by the customer may be accepted as a commercial risk, and the customer may be allowed to return unwanted goods.

4.10 Summary of section

In this section, we described the different types of non-functional requirements and their main characteristics, and illustrated each type of non-functional requirement. We looked in more detail at security requirements and the issues they raise.

5 Conformance testing and fit criteria

On completion of this section you should be able to:

- understand the concept of, and the need for, fit criteria;
- suggest fit criteria for different types of requirement.

The software product that we develop should satisfy all of the functional and non-functional requirements we identify. In other words, the software product must conform to its requirements. To find out if this is the case (in the absence of a formal proof, that is, a rigorous mathematical argument), we need to test the product against its requirements, that is, perform conformance testing. However, for us to be able to do this, each requirement needs to be expressed in such a way that it can indeed be tested. We will therefore consider what criteria can be added to each functional and non-functional requirement, to allow us to tell whether it has been satisfied. Each fit criterion makes precise some aspect of a requirement.

A *fit criterion* is a precise and testable statement of a requirement. It may specify a quantitative measure for some aspect of the system's behaviour or performance, or define some other quality that the system must possess if it is to meet the requirement. Whatever form it takes, it must be capable of being tested in an objective fashion.

The idea is for each requirement to have a quality measure that makes it possible to divide all solutions to the requirement into two classes: those for which we agree that they fit the requirement and those for which we agree that they do not fit the requirement.

(Alexander, 1964)

The process of attaching a fit criterion to a requirement helps to clarify the requirement; determining the fit criteria will also involve interacting with the client and other stakeholders to check that the criteria being specified are acceptable and reflect the correct understanding of the requirement. Specifying fit criteria can, therefore, be a helpful communication or negotiation tool for interacting with different stakeholders. When a designer addresses a set of requirements, they should fit each part of the solution to the relevant requirement; that is, they should ensure that each part of the solution satisfies the relevant requirement.

The fit criteria for a functional requirement need to be written in such a way that we can tell whether the product satisfies the requirement. In other words, they need to tell us whether the system succeeds in fulfilling the requirement. For example, consider the following requirement.

> The system shall accept a credit card number from a client.

The following fit criterion might be suitable.

> A valid credit card number has been stored in the system.

The fit criterion for a non-functional requirement needs to be expressed in terms of some measurable quantity; that is, some scale of measurement needs to be used.

A scale of measurement is the unit against which the conformance of the product is tested. It is the unit used in defining the fit criterion. For example, a usability requirement of 'time taken to complete a task' has the scale of time – microseconds, seconds, minutes and so on.

To find an appropriate scale of measurement, you need to analyse the description and rationale of the requirement to derive a fit criterion. The scale of measurement is the unit which is specified as a part of the fit criterion.

For example, consider the following non-functional requirement associated with the above functional requirement.

> The credit card number should be accepted securely.

The following fit criterion might be appropriate.

> The credit card number should be revealed to a third party in less than 0.0001% of cases.

In this case the scale is implicitly the number of cases when the credit card number is used.

The following non-functional requirement contains its own fit criterion, as it is already expressed in terms of the quantity time.

> The credit card number should be accepted within five seconds.

As well as fit criteria for requirements, it is also useful to have fit criteria for groups of related requirements, for example, all of the requirements associated with one use case. Recall that a use case is a description of how a system is to be used for a specific goal. The fit criteria for a use case specify the measurable goal or outcome of the use case. Often the initial fit criteria will be obtained by observing the way any current system operates or discussing with users how they feel the new system should perform.

A key use of the fit criteria is in testing the system against its requirements. It is good practice for requirements analysts to specify fit criteria by involving testers early on in the development life cycle. In fact, the fit criteria can be specified as the requirements are discovered and clarified. The testers act as consultants to specify the fit criteria. Testers can, from their experience, help to identify whether something needs to be tested, or indeed *can* be tested, and whether a suggested fit criterion is an appropriate measure of the requirement.

The fit criteria are like benchmarks, or goals, with which the testers can compare the delivered product or solution against the original requirement. The fit criteria are useful for developers, as they develop the product to meet the standard set by the fit criteria. You cannot successfully design a solution to the requirement if you have no way of knowing whether the solution will meet the requirement. Fit criteria are acceptance criteria for the client to accept the product against the requirements. As such, they may form part of the contract between the client and the developers.

Where an existing system is being replaced, it can be helpful in eliciting fit criteria, and may provide the basis for obtaining empirical data on the current situation. Observing users' current work practices gives an idea of the current quality of the system. Talking to the users gives an idea of the expectations they have from the future system.

It might not be possible to specify the fit criteria when you first elicit a requirement. Depending upon the requirements, the clarification of requirements and fit criteria may involve visiting users and observing their current work practices, or interviewing users; or it may involve meeting up with domain experts, determining business and resource constraints or trade-offs between requirements, and so on.

There can be situations when, having elicited a requirement, it turns out to consist of several requirements. So each of these requirements will have to have its own fit criteria. It could also be that the requirement might not have been properly thought through, and while determining the fit criteria and clarifying them with the users or client, it may transpire that the suggested requirement is not a requirement after all and should be deleted from the specification.

5.1 Examples

We now look at some further examples drawn from a range of systems such as an ATM machine, websites, an information kiosk and so on. These illustrate that a range of ways of tackling fit criteria are appropriate, and that there is some scope here for creativity!

Example 3 Product failure

Description: The ATM machine shall allow the customer to make a withdrawal as fast as possible.

The fit criterion can be determined by asking the client: what would be considered a failure to meet this requirement? The client might say that customers will abandon the transaction if they are unable to make the withdrawal within 30 seconds, and therefore 30 seconds is the acceptable time limit. The fit criterion might be as follows.

Fit criterion: 90% of experienced customers should spend no more than 30 seconds to make a withdrawal.

The definition of 'experienced' customers will need to be clarified with the client. For example, experienced customers could be those who use the ATM machine at least once a fortnight. Also, the acceptability of the 90% success rate to the client will have to be checked.

Example 4 Look-and-feel requirement

Description: The student website of the M363 course shall have a similar look and feel to other OU course websites.

Fit criterion: The M363 course website shall conform to the OU web design and accessibility guidelines.

This fit criterion would be used by the Quality Assurance group to test the website before it goes live.

Example 5 Usability requirement

For a public information kiosk at a railway station, the usability requirement is as follows.

Description: The kiosk shall be usable by a member of the public who may not speak or read English.

One possible fit criterion is as follows.

Fit criterion: 15 out of 20 non-English speakers/readers shall be able to use the kiosk without either referring to the online help or walking away without the desired information.

Example 6 Performance requirement

Description: An existing customer shall be able to complete the check out process (specifying the delivery address and delivery mode) on the ecommerce website as quickly as possible.

Fit criterion: 90% of existing customers should spend no more than 30 seconds to complete the check out process.

However, in specifying this fit criterion, you will need to find out what is meant by an 'existing customer'. An existing customer could be one who has shopped on the site before and whose credit card details and preferred addresses for delivery are already held in a personalised account on the ecommerce website. You will also need to check whether 90% is acceptable to the client. Finally, 30 seconds should be an acceptable time limit, with some evidence to back it up – such as observations of existing customers on the website (if the site is already operational and an update is being planned), or data from the web log.

Example 7 Operational requirement

Description: The railway ticketing machine shall be usable in a very humid environment.

Fit criterion: The product shall function correctly even when exposed to a humidity of greater than 80% for up to 48 hours at a time.

Example 8 Maintainability requirement

Description: The websites of OU courses shall be updated frequently.

Fit criterion: Updates to OU course websites shall be made weekly and be visible to students within 8 hours of being loaded.

Example 9 Security requirement

Description: The product procurement prices shall only be accessible to warehouse staff.

Fit criterion: Non-warehouse staff shall be unable to access product procurement prices.

Example 10 Cultural and political requirement

Description: The terminology and icons on student websites of the OU shall not be UK-centred.

Fit criterion: Student websites of the Open University must conform to the university's web design guidelines for internationalisation.

Example 11 Legal requirement

Description: The course material on the M363 course website shall have all its copyrights cleared before it goes live.

Fit criterion: The Rights Department has obtained clearance for all copyright material on the site.

SAQ 16

(a) What is the first step towards finding whether a solution fits a requirement?

(b) What is a fit criterion?

ANSWER.....

(a) The first step towards finding whether a solution fits a requirement is to attach a quantifiable measure to the requirement so that it is testable.

(b) A fit criterion is a quantification or measurement of the requirement such that the design solution can be measured to find if it unambiguously meets the requirement.

SAQ 17

(a) Who needs the fit criteria?

(b) When are fit criteria specified?

ANSWER.....

(a) The developers of the product use the fit criteria to develop the product to meet those criteria. The testers use the fit criteria to determine whether the delivered product meets the original requirements. The clients for whom the product is being developed use the fit criteria as acceptance criteria for the product.

(b) The fit criteria can be written or elicited as the requirements are elicited, for example, once use cases have been drawn up and we find the requirements for each task in each use case.

SAQ 18

(a) What is a fit criterion for a functional requirement?

(b) Do the fit criteria of functional requirements have scales of measurement?

(c) Does a fit criterion indicate how the functional requirement would be tested?

ANSWER.....

(a) A fit criterion for a functional requirement specifies the completion of the function of the product that is specified by the functional requirement. For example, if the required function is to send an email to the student after a marked TMA has been uploaded by the tutor, then the fit criterion for this requirement is that an email should indeed be sent to the student and reflected in their mailbox on the forum.

(b) No. The fit criteria of functional requirements do not have scales of measurement. Success is given in terms of a yes/no answer that implies that the required function is either achieved or not.

(c) A fit criterion provides some target that, when the solution is tested, reveals whether the solution conforms to the requirement. The fit criterion does not indicate how the product will be tested. It merely states that the tester should ensure that the product complies with the specified fit criterion.

SAQ 19

What does the fit criterion for a non-functional requirement specify?

ANSWER

A fit criterion for a non-functional requirement specifies a value or values, on a particular scale of measurement, that must be attained by the property or quality with which the requirement is concerned.

Exercise 7

How might you determine the fit criterion for the following performance requirement?

> The customer services personnel should be able to respond rapidly to customer queries.

Solution

For this performance requirement, talking to the users about their own expectations and those of their customers might reveal that the preferred task time is 1 minute, and that currently it takes three minutes to answer a customer query because of a cumbersome search facility on the current system. So the fit criterion for this requirement might be something like the following.

> The customer services personnel must be able to find an answer to a customer's query in not more than one minute of a customer's call.

This example shows that the fit criterion is backed by empirical data for the current situation.

Exercise 8

Here is an example of a situation and a possible fit criterion for a requirement to address it.

Situation: Customers send letters to the customer services department complaining about unsatisfactory experiences with a particular product.

Fit criterion: The number of letters complaining about the product within a defined time period falls below a set threshold.

Give suggestions of possible fit criteria for requirements to address the following situations related to performance requirements.

(a) Customers return the product if they are unhappy with it after a purchase has been made.

(b) Users do not find the system user-friendly, because of the number of errors that are made in completing a task.

(c) The staff members are fairly satisfied, and fewer of them are leaving the organisation.

(d) The hotel customers find the atmosphere in the hotel very restful.

Solution

(a) The number of product items returned within a particular period as a percentage of those sold in the same period falls below a set threshold.

(b) Errors shall not occur on more than a specified proportion of the occasions on which the task is executed.

(c) The staff turnover does not exceed a specified level.

(d) The percentage of customers staying in the hotel, over a particular period, who agree that the hotel is restful is above a defined level.

Exercise 9

We have listed below some requirements and their fit criteria from the Requirements Tool in Exercise 4. Assess the suitability and clarity of the fit criteria.

(a) Requirement: The product shall enable the user to view a summary of all requirements.

Fit criterion: The product should show a summary of all requirements.

(b) Requirement: The product shall show all requirements that conflict with a given requirement.

Fit criterion: The product should show all conflicts with a given requirement.

(c) Requirement: The product shall perform a range of checks on the set of requirements.

Fit criterion: The product should display the results of a range of checks.

Solution

(a) The fit criterion does not make it clear what the 'summary' consists of. Also, it does not make it clear whether it's the summary of each requirement or a 'composite' summary of all the requirements.

A better fit criterion might be: 'A list will be produced giving a one line summary for each requirement.' This might mean we need to enter this in the form of a name for each requirement, for example, 'Accept a Reservation', at an earlier stage.

(b) It's not clear from the fit criterion whether the requirements that conflict with one requirement need to be shown, or whether it's the nature of the conflicts that needs to be shown.

A better fit criterion might be: 'For a given requirement, state the number of any requirement that conflicts, along with the reason.'

(c) The requirement and the fit criterion do not make it clear what the 'range of checks' are nor what the 'result' should consist of.

A better fit criterion might be: 'The requirements will be checked, and the number of any requirement where all fields are not correct will be reported, as well as the number of any requirement that conflicts with any other.'

Exercise 10

Are fit criteria always attainable?

Solution

No. Fit criteria are not always attainable. Whether they are actually attainable depends on what happens during the development cycle. Constraints can arise when attempting to achieve the fit criteria owing to, for example, the product's operating environment, the client's budget, or time. Measurements of fit criteria can only be made in reasonable ways and in reasonable times.

5.2 Summary of section

In this section, we discussed the need for, and the concept of, fit criteria, and applied it to different types of functional and non-functional requirements.

6 Representing the requirements

On completion of this section you should be able to describe and use the Volere template for representing requirements.

Requirements need to be documented within a project. There are many reasons why this is so. Requirements documents are an aide-memoire to record decisions agreed among the system's stakeholders. They are the starting point from which the system is designed and built, and are the basis for the validation of the system, once it is built. Documentation may take many forms. The one we adopt here is based on the Volere approach by Robertson and Robertson (2006).

6.1 The Volere template

The **Volere template** is a template for a document that collates all the requirements of a system, together with other issues that may affect those requirements. The template provides a sort of container, in which the information can be organised systematically. The following text describes the template in its entirety, although you will only use parts of it.

Note that a requirements document is organic. It grows and changes throughout the duration of a project. The Volere template supports a disciplined way of recording requirements as they are discovered and refined. The template is organised into four main sections, each including a number of requirements categories.

Volere template

Product constraints

These are restrictions and limitations that apply to the project and the product. In this section of the template you will record the following types of information.

1 *Purpose of the product*. This is the reason for building the product, and the business advantage if it is done. It describes the problem faced by a customer, or a business opportunity a customer wants to exploit, and it explains how the product is going to help. For instance:

   We are expanding our business internationally; our current billing system is based on pounds sterling; we need a new system that can deal with foreign currency.

2 *Customer and other stakeholders*. These are the people and organisations, other than the users, with a vested interest in the product.

3 *Users of the product*. These are the intended end-users of the product. Their description should also include how they affect the product's usability requirements. For instance:

   The users of the system are potential passengers interested in train timetables and journey information. We cannot assume they are in any way familiar with the technology; they should be able to use the system with hardly any support.

4 *Requirements constraints*. These are constraints on the way the product must be designed or on the project itself. For instance, they could state how the product should look or which technology it should comply with; or they could

place restrictions on the development budget and time. Note that these are not requirements per se, but they put restrictions on the requirements: only requirements that can be satisfied within the given constraints can actually be addressed by the project. Typically, constraints are raised by management policies.

5 *Naming conventions and definitions*. This is the vocabulary of the project, including terms and definitions that are commonly used in communication with the stakeholders. A more formal companion to this, the glossary, is also introduced incrementally in the project to include the definition of all relevant and technical terms.

6 *Relevant facts*. These are external facts that may have an effect on the product, but do not necessarily translate into requirements. For instance, for a system that should monitor and forecast the de-icing of frozen roads in certain geographical regions, a relevant fact could be:

One ton of de-icing material is required to de-ice 2 km of single-lane carriageway.

7 *Assumptions*. These are assumptions made by the project team. They may concern the operational environment of the product, as well as its performance or capacity. Here are some examples:

No more than 100 users will be connected to the system at any one time.

The system will sample the road-surface temperature every 200 milliseconds.

Functional requirements

These cover the functionality of the product. They include the following.

8 *Scope of the product*. This defines the product boundaries. To be able to set the product boundaries, you first need to understand the context of the system, that is, the working environment in which it will operate.

9 *Functional requirements*. These are what the product must do to contribute to the product goal. They could be functions or actions the product must perform. For instance, the following is a function.

The product should record road weather conditions.

But the following is an action.

The product should alert the operator when the temperature falls below 0 °C.

Non-functional requirements

These are the properties the product must have. They include the following.

10 *Look-and-feel requirements*. The intended appearance of the product.

11 *Usability requirements*. These are based on the intended users.

12 *Performance requirements*. How fast, accurate, reliable etc. the product must be.

13 *Operational requirements*. The product's intended operating environment.

14 *Maintainability and portability requirements*. How easily the product can change.

15 *Security requirements*. The security, confidentiality and integrity of the product.

16 *Cultural and political requirements*. Human factors that may affect the product.

17 *Legal requirements*. Conformance of the product to applicable laws.

Project issues

These are not requirements, but concerns that are brought to light by the requirements activities. They appear in the requirements document because they help to clarify the requirements further.

18 *Open issues*. These are issues that have arisen from the requirements activities, but are not yet resolved. These issues may have to do with changes that have occurred in the business, the requirements or the project.

19 *Off-the-shelf solutions*. This is a record of ready-made components that might be used in the project, and their impact on the product requirements.

20 *New problems*. This category is used to highlight possible problems caused by the introduction of the product, for instance, changes which are required in current working practices or the need for special training.

21 *Tasks*. This category highlights the effort required to deliver the product, whether it is a complete build or the acquisition of existing off-the-shelf solutions.

22 *Cutover*. These are things that need to be done after installing the product, but before it can work successfully. Typically, this covers tasks to convert data or processes from existing systems.

23 *Risks*. These are the risks the project is likely to face, together with strategies for coping, should they become problems at some point during the project.

24 *Costs*. This is an estimate of the cost or effort needed to build the product.

25 *User documentation*. This is the plan for writing the user instructions and documentation.

26 *Waiting room*. This is for requirements that for some reason cannot be included in the current release of the product, but might be included in future releases.

Source: http://www. volere.co.uk/ [accessed: 27 March 2007]

Functional and non-functional requirements are the core of the requirements document. While all other sections of the template can be filled in as free-form text, the Volere approach recommends a more formal recording of functional and non-functional requirements.

The following information should be recorded for each functional/non-functional requirement:

- requirement number – a number uniquely identifying the requirement;
- events/use cases – the use cases or events that are associated with this requirement;
- description – a one-sentence statement of the intent of the requirement;
- rationale – a statement of why the requirement is considered important or necessary;
- source – a record of who raised the requirement in the first instance;
- fit criteria – acceptance criteria, written in a quantified manner so that the solution can be tested against the requirement;
- dependencies – the identifying numbers of other requirements that have an impact on this one;
- conflicts – the identifying numbers of requirements that contradict this one, or make this one less feasible;
- history – the origin of the requirement, and any changes to it.

There are some notable entries in this list. First, history indicates that requirements do not remain unchanged during the development process. Requirements are recorded, refined and even discarded, usually through a process of analysis and negotiation with the system's stakeholders. Second, dependencies and conflicts indicate that

requirements do not exist in isolation, but generally relate to other requirements, by being either dependent on them or in conflict with them. This means that a change to a requirement may need to be propagated to other requirements, and that conflicts need to be identified and resolved.

The complexity of the Volere template is a clear indication that managing the requirements of realistic systems is not a trivial task. Tracking requirements and their changes throughout a project is vital to the success of the project, as a customer's satisfaction depends on it. In large projects, the use of CASE tools for requirements management is beneficial, if not essential.

SAQ 20

What is the advantage of capturing requirements using a template, rather than adopting ones own format?

ANSWER.....

The template is divided into a fixed set of categories, which means we are less likely to forget some types of requirement. It also saves us from having to work out what categories of requirement to deal with each time we start a new document. It helps us to communicate requirements to our fellow-developers. If we have a standard template, then everyone will know what information to expect and in what order. (It might also allow us to compare projects and even reuse requirements more easily.)

6.2 Summary of section

In this section, we introduced the Volere template for the description of requirements. You will see a detailed example of how the Volere template is used in *Unit 4*.

7 Summary

In this unit you have seen the importance of carefully documented requirements. You have seen the differences between functional and non-functional requirements and the ways in which each of these can be recorded. The importance of attaching measurable fit criteria to each requirement has been established. Finally, you saw how we can make use of the Volere template to help ensure we cover all the categories of requirements.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- understand the nature and importance of requirements;
- describe the activities of a requirements engineering process;
- use a requirements template to write a requirements document in a structured way;
- define and distinguish between functional and non-functional requirements;
- describe the various types of non-functional requirement;
- produce clear functional and non-functional requirements;
- produce measurable, and hence testable, functional and non-functional requirements;
- discuss the relationship between requirements and testing.

References

Alexander, C. A. (1964) *Notes on the Synthesis of Form*, Cambridge, MA, Harvard University Press.

Alexander, I. F. and Stevens, R. (2002) *Writing Better Requirements*, London, Addison-Wesley.

Boehm, B. and In, H. (1996) Identifying quality-requirement conflicts. *IEEE Software*, vol.13, no. 2, pp. 25–36.

Brooks, F. P. (1987) No Silver Bullet: Essence and Accidents of Software Engineering, *Computing*, vol. 20, no. 4, pp. 10–19.

Pugh, D. (1993) 'Understanding and managing organisational change' in Mabey, C. and Mayon–White, B. (eds) *Managing Change* (2nd edn), Buckingham, The Open University.

Robertson, S. and Robertson, R. (2006) *Mastering the Requirements Process* (2nd edn), Reading, MA, Addison-Wesley.

Somerville, I. (2004) *Software Engineering* (7th edn), Addison-Wesley.

The Standish Group (1994) *The CHAOS Report* [online], http://www.standishgroup.com/sample_research/chaos_1994_1.php (Accessed 12 March 2007).

Acknowledgements

Grateful acknowledgement is made to the following sources.

Figures

Figure 1: © 2001 United Feature Syndicate, Inc.

Figure 2: Robertson, S. and Robertson, R. (1999) Mastering the Requirements Process, Addison Wesley.

Text

Volere Template: We acknowledge that this document uses material from the Volere Requirements Specification Template. Copyright © 1995–2006 Atlantic Systems Guild Limited.

Index